

InferenceChain: A Decentralized AI Inference Network

Technical Whitepaper — v0.5 (April 2026)

Abstract

InferenceChain is a permissioned blockchain designed to decentralize deep neural network (DNN) inference. It implements three novel protocols developed in the author's own research: **Quality of Inference (QoI)**, **Proof of Quality of Inference (PoQI)**, and **Scalable Byzantine Fault Tolerant consensus (S-BFT)**. Unlike general-purpose blockchains, InferenceChain's consensus mechanism is inference itself — nodes prove their honesty by producing outputs that agree with a verified majority, and are rewarded or slashed accordingly. The network uses the **INFER** token for staking, rewards, and fees.

1. Motivation

Centralized AI inference has fundamental problems:

- **Trust** — users cannot verify that a model ran correctly or that results were not manipulated
- **Single point of failure** — one provider going offline stops all dependent systems
- **Accountability** — there is no on-chain record of what model produced what output

Existing blockchains cannot address this because they have no mechanism to evaluate the *quality* of computational work — only whether computation happened. InferenceChain solves this by making quality measurable through Byzantine-robust agreement among independent DNN nodes.

2. Architecture Overview

InferenceChain runs as a network of nodes, each of which may be one of two types:

Node Type	Role	Stake Requirement
DNN Node	Runs inference, participates in QoI rounds, earns inference rewards	≥ 1,000 INFER
PoS Node	Validates simple transactions, seals PoS blocks	≥ 500 INFER

All nodes participate in the Proof-of-Stake (PoS) layer. Only DNN nodes that have passed **Proof of Model (PoM)** verification are eligible to participate in QoI consensus rounds.



3. Blockchain Layer

3.1 Block Types

InferenceChain produces two block types per slot:

- **PoS Block** — contains simple transactions (transfers, staking, node registration). Produced by an elected PoS proposer using stake × reputation weighted selection.
- **QoI Block** — produced when an inference request is pending and DNN validators are available. Contains the inference transaction, all node responses, consensus result, rewards, and slashes.

3.2 Transaction Types

Family	Type	Description
SIMPLE	<code>token_transfer</code>	Transfer INFER between addresses
SIMPLE	<code>stake</code>	Lock INFER as validator stake
SIMPLE	<code>unstake</code>	Withdraw staked INFER
SIMPLE	<code>node_register_dnn</code>	Register a DNN validator with model commitment
SIMPLE	<code>node_register_pos</code>	Register a PoS-only validator
SIMPLE	<code>model_response</code>	Node's answer to a PoM challenge
INFERENCE	<code>inference_request</code>	Client submits an image for inference
SYSTEM	<code>inference_response</code>	Node's inference result (per QoI round)
SYSTEM	<code>consensus_result</code>	Final agreed class + confidence
SYSTEM	<code>reward</code>	Minted INFER for honest nodes

SYSTEM	<code>slash</code>	Burned stake for Byzantine nodes
SYSTEM	<code>node_admitted</code>	Node passed PoM, activated
SYSTEM	<code>node_rejected</code>	Node failed PoM, deactivated

3.3 Address Derivation

```
private_key → secp256k1 → public_key (64 bytes uncompressed)
address = SHA-256(public_key_bytes)[:20] as 40-char hex
```

This is identical in the Python backend (`core/node/identity.py`) and the JavaScript wallet (`frontend/src/utils/wallet.js`).

3.4 Transaction Signing

Each transaction has a canonical ID computed as:

```
tx_id = SHA-256(JSON({
  tx_type, sender, recipient, payload, nonce, fee, timestamp
}, sort_keys=True, no_spaces))
```

The sender signs `SHA-256(tx_id.encode())` with their secp256k1 private key using compact (64-byte) ECDSA. The public key is included in the wire format so the backend can verify it and register it in the on-chain public key registry (`ChainState.pubkeys`).

3.5 Persistence

The chain is persisted to a SQLite database (`data/chain.db`) using WAL mode. On startup, the node replays all committed blocks to reconstruct chain state. The state snapshot (including the public key registry) is saved after every committed block.

4. Proof of Model (PoM)

Before a DNN node can participate in QoI consensus, it must prove its model is genuine. The PoM protocol works as follows:

- Registration** — Node submits `node_register_dnn` tx with: model name, architecture spec, `SHA-256(weights_file)`, dataset ID, endpoint URL, public key.
- Challenge** — Protocol issues a `model_challenge` system tx with 50 deterministic test indices from the CIFAR-10 test set.
- Response** — Node submits `model_response` tx with argmax predictions for the 50 samples.
- Verification** — Active validators independently verify: download weights from node's endpoint, compute `SHA-256(weights)`, compare to on-chain commitment, run inference on the same 50 samples, submit `model_verify` tx.
- Admission or Rejection** — When $2f+1$ positive verifications are collected, a `node_admitted` system tx activates the node. Below the accuracy threshold (80%), `node_rejected` is emitted.

Minimum model accuracy: **80%** on the challenge set.

5. Quality of Inference (QoI / PoQI)

QoI is the core consensus mechanism. It is a PBFT-style protocol adapted for DNN inference.

5.1 Round Lifecycle

```
IDLE → PRE_PREPARE → PREPARE → COMMIT → COMMITTED  
■ (on timeout) VIEW_CHANGE → NEW_VIEW
```

1. **PRE_PREPARE** — The elected primary broadcasts its inference result (class probabilities) for the image.
2. **PREPARE** — Each replica runs the same image through its own model and broadcasts its result.
3. **COMMIT** — Once $2f+1$ PREPAREs agree on a consensus class (majority vote), nodes broadcast COMMITs.
4. **COMMITTED** — Once $2f+1$ COMMITs are received, the round finalizes. A `consensus_result` tx is written to chain.

5.2 Quality Scoring

Nodes are scored based on how close their softmax output is to the consensus class prediction. The primary metric is cosine similarity between a node's probability vector and the consensus vector. Honest nodes receive INFER rewards proportional to their QoI score; nodes that deviate beyond the Byzantine threshold are slashed.

Outcome	Effect
Honest (argmax = consensus class)	INFER reward + reputation gain
Byzantine (wrong argmax)	SLASH_PENALTY (100 INFER) stake deduction + reputation loss
Primary (drove consensus)	Additional LEADER_BONUS (2 INFER) + rep bonus

5.3 Parameters

Parameter	Value
Inference reward per round	10 INFER
Slash penalty	100 INFER
Leader bonus	2 INFER
Minimum confidence	0.5
Reputation cap per round	+0.5
Reputation penalty	-0.3

6. S-BFT: Scalable Byzantine Fault Tolerance

6.1 The Scaling Problem

With N DNN nodes in the network, running QoI across all of them for every inference request is unscalable:

- **Message complexity** grows as $O(N^2)$ per round
- **Latency** tracks the slowest node in the full set
- **Relevance** — most nodes may not be trained on the requested model/dataset

6.2 Quorum Selection

S-BFT solves this by electing a small **quorum** of K nodes per inference request. The quorum is:

1. **Filtered** — only nodes whose `model_name` and `dataset_id` match the request
2. **Randomly sampled** — without replacement, from the filtered eligible pool
3. **Deterministic** — the seed is `SHA-256(block_hash || request_id)`, reproducible by any participant
4. **Verifiable** — any node can recompute the same quorum from public information

```
seed = SHA-256(block_hash + request_id)
quorum = random.sample(eligible_filtered_nodes, K, seed=seed)
```

6.3 Quorum Parameters

Parameter	Value	Notes
Target quorum size	10	Ideal size
Floor	4	Minimum for f=1 (need $\geq 3f+1$)
Ceiling	19	Maximum (gives f=6)

If the filtered pool has fewer than 4 eligible nodes, the quorum falls back to all eligible DNN validators.

6.4 Local BFT Parameters

Each quorum operates with its own fault-tolerance parameters, independent of the global chain f:

```
f = (K - 1) // 3
threshold = 2f + 1

K=4 → f=1, threshold=3
K=10 → f=3, threshold=7
K=19 → f=6, threshold=13
```

6.5 Non-Quorum Nodes

Nodes not elected to the quorum for a given round stand by for one block slot and do not participate. The committed QoI block arrives via P2P gossip and is ingested normally.

6.6 Quorum Integrity at Commit

When `_commit_qoi_block` executes, it validates that every commit signature came from a node that was in the elected quorum. Signatures from outside the quorum are stripped — those nodes receive no reward and their response is ignored.

6.7 OOD-Biased Quorum Selection (Knowledge Self-Assessment)

Every admitted DNN node automatically calibrates an **Out-of-Distribution (OOD) scorer** the moment its `NODE_ADMITTED` transaction is committed in a PoS block. The scorer measures how *in-distribution* (familiar) a candidate image is for a given node's model — enabling the quorum to be biased toward validators that are most capable on the specific input domain.

Scorer hierarchy

Scorer	Method	Calibration cost
<code>MahalanobisOODScorer</code>	Class-conditional Gaussian fit on penultimate-layer features	One calibration pass over CIFAR-10 test set
<code>EnergyOODScorer</code>	Free-energy of logits: $-T \cdot \log \sum \exp(f_{\square}/T)$	Zero — uses raw logits
<code>LikelihoodRegretScorer</code>	Paper-exact VAE Likelihood Regret	Separate VAE training pass

At admission time, nodes attempt Mahalanobis calibration first and fall back to the Energy scorer if the CIFAR-10 test set is not available on disk. The calibration runs in a background thread so it never blocks the consensus loop.

Biased selection algorithm

```
# 1. For each eligible node, query its OOD scorer with the request image
scores = {node_id: scorer.score(image_tensor) for node_id in eligible}

# 2. Build a top-2K shortlist (lowest score = most in-distribution)
shortlist = sorted(scores, key=scores.get)[:2 * K]

# 3. Sample K nodes deterministically from the shortlist
seed = SHA-256(block_hash + request_id)
quorum = random.sample(shortlist, K, seed=seed)
```

Nodes without a calibrated scorer fall back to the old uniform-random pool, so the feature degrades gracefully on new or partially-admitted networks. BFT determinism is fully preserved: any peer can reproduce the same quorum from public information.

7. LLM Orchestration Layer

The LLM layer is an **optional, consensus-independent** coordination service. It runs above the blockchain, never participates in QoI consensus or quorum selection, and can be omitted entirely without affecting network safety.

7.1 Architecture

```
client → POST /llm/* → InferenceOrchestrator → LLM Provider
reads chain state (read-only)
never writes transactions
```

The `InferenceOrchestrator` is initialized at node startup and injected into the FastAPI application as a singleton dependency. If no provider is configured, all `/llm/*` endpoints return HTTP 503.

7.2 Provider Configuration

Flag	Variant	Notes
<code>--llm-provider ollama</code>	Local Ollama server	Recommended — no API key, fully offline
<code>--llm-provider openai</code>	OpenAI API	Requires <code>OPENAI_API_KEY</code> or <code>--openai-key</code>
<code>--llm-provider mock</code>	Deterministic mock	CI / unit tests
(omitted)	Disabled	Default — no LLM, no extra dependencies

7.3 Endpoints

Endpoint	Body	Response
POST <code>/llm/route</code>	<code>{"image_description": "..."} </code>	<code>{model_hint, dataset_id, reasoning}</code>
POST <code>/llm/analyse</code>	<code>{}</code>	<code>{anomalous_nodes, recommend_pom, summary}</code>
POST <code>/llm/decompose</code>	<code>{"task_description": "..."} </code>	<code>[{sub_task, model_hint, priority}]</code>
POST <code>/llm/explain</code>	<code>{"question": "..."} </code>	<code>{answer, sources}</code>

9. Wallet & Client Protocol

7.1 Key Scheme

- **Mnemonic** — 12 BIP39 words (128-bit entropy, [@scure/bip39](#))
- **Private key** — first 32 bytes of BIP39 seed (`mnemonicToSeedSync().slice(0, 32)`)
- **Public key** — 64-byte uncompressed secp256k1 point (no 04 prefix)
- **Address** — `SHA-256(pubkey_bytes)[:20]` as 40-char hex

7.2 Storage Security

The private key is **never stored in plaintext**. LocalStorage holds only:

```
{
  "address": "...",
  "publicKey": "...",
  "encrypted": {
    "ciphertext": "...",
    "salt": "...",
    "iv": "..."
  }
}
```

Encryption: **AES-256-GCM** with **PBKDF2** key derivation (100,000 iterations, SHA-256). The private key lives only in React state while the wallet is unlocked.

7.3 Transaction Flow

1. Wallet builds tx locally (canonical JSON, computes tx_id)
2. Signs: `ECDSA(SHA-256(tx_id.encode()), private_key)` → 64-byte compact sig
3. Submits to `POST /tx/submit` with `public_key` field included
4. Backend registers pubkey in `ChainState.pubkeys` on first confirmed tx
5. Subsequent txs from the same address are verified using the registered pubkey

10. Token Economics

Parameter	Value
Token name	INFER
Maximum supply	100,000,000 INFER
Genesis allocation	10,000,000 INFER
Min DNN validator stake	1,000 INFER
Min PoS validator stake	500 INFER
Slash penalty	100 INFER per Byzantine act
Inference reward	10 INFER per consensus round
Block fee	Distributed to the block proposer

11. Network Protocol

Nodes communicate over WebSocket (`ws://host:port+1000`). Message types:

Message	Direction	Purpose
POS_BLOCK	Proposer → Peers	Candidate PoS block
POS_VOTE	All → All	Vote on candidate block
QOI_MSG	Quorum → Quorum	PRE_PREPARE / PREPARE / COMMIT / VIEW_CHANGE
NEW_BLOCK	Committer → Peers	Committed block for ingestion
IMAGE_REQUEST	DNN node → Peers	Request image bytes by hash
IMAGE_RESPONSE	Peer → DNN node	Image bytes

12. REST API

Base URL: <http://localhost:8000>

Endpoint	Description
GET /chain/height	Current block height
GET /chain/tip	Latest block
GET /chain/block/{height}	Block by height
GET /chain/blocks	Paginated block list
GET /chain/stats	Chain statistics
GET /state/balance/{address}	INFER balance
GET /state/history/{address}	Transaction history
GET /state/nodes/active	Active validators
GET /state/nodes/dnn	Active DNN validators
GET /state/nodes/pos	Active PoS validators
GET /tx/{tx_id}	Transaction status
GET /tx/pending	Mempool snapshot
POST /tx/submit	Submit signed transaction
POST /state/faucet	Testnet token faucet
GET /p2p/peers	Known peers
GET /dashboard/validators	Validator dashboard data
GET /dashboard/stats	Network statistics
POST /llm/route	(opt-in) Recommend model + dataset for an image description
POST /llm/analyse	(opt-in) Scan validators for anomalies
POST /llm/decompose	(opt-in) Split a task into parallel sub-requests
POST /llm/explain	(opt-in) NL Q&A; about chain state

WebSocket streams:

- [ws:///.../ws/consensus-rounds](#) — live consensus events
 - [ws:///.../ws/validators](#) — validator state changes
 - [ws:///.../ws/models](#) — model registry events
-

13. Running a Node

Single node (default, persistent)

```
python node_runner.py
# Persists to data/chain.db automatically
# REST API: http://localhost:8000/docs
```

Two-node testnet

```
# Terminal 1
python node_runner.py --port 8000 --db-path data/node1.db

# Terminal 2
python node_runner.py --port 8001 --db-path data/node2.db \
--peer http://127.0.0.1:8000
```

DNN node

```
python node_runner.py \
--node-type dnn \
--model resnet20 \
--weights-dir models/weights \
--private-key <hex_private_key>
```

DNN node with LLM layer (local Ollama)

```
ollama pull llama3.2
python node_runner.py \
--node-type dnn \
--model resnet20 \
--weights-dir models/weights \
--private-key <hex_private_key> \
--llm-provider ollama --llm-model llama3.2
```

DNN node with LLM layer (OpenAI)

```
export OPENAI_API_KEY=sk-...
python node_runner.py \
--node-type dnn \
--model resnet20 \
--weights-dir models/weights \
--private-key <hex_private_key> \
--llm-provider openai --llm-model gpt-4o-mini
```

In-memory (ephemeral, no persistence)

```
python node_runner.py --db-path ""
```

14. Current Implementation Status

Component	Status
Blockchain core (blocks, txs, chain state)	✓ Complete
Proof-of-Stake consensus	✓ Complete
Proof of Model (PoM) verification	✓ Complete
QoI / PoQI consensus state machine	✓ Complete
S-BFT quorum selection (hash-based)	✓ Complete
Signature verification (wallet users)	✓ Complete
SQLite persistence (WAL)	✓ Complete
P2P gossip network	✓ Complete
REST API (all endpoints)	✓ Complete
React frontend dashboard	✓ Complete
secp256k1 wallet (BIP39, AES-GCM)	✓ Complete
CIFAR-10 CNN model suite (7 architectures)	✓ Complete
OOD scoring (Mahalanobis / Energy / LikelihoodRegret)	✓ Complete
OOD auto-calibration on NODE_ADMITTED	✓ Complete
OOD-biased quorum selection (top-2K shortlist)	✓ Complete
LLM orchestration layer (OpenAI / Ollama / Mock)	✓ Complete
LLM /route /analyse /decompose /explain endpoints	✓ Complete
Wallet auto-lock (idle timer)	■ Pending
Faucet as real SYSTEM tx	■ Pending

15. References

1. D. Papaioannou et al., *Quality of Inference: A Protocol for Byzantine-Robust DNN Consensus*, AUTH, 2024.

2. D. Papaioannou et al., *S-BFT: Scalable Byzantine Fault Tolerance via Per-Request Quorum Selection*, AUTH, 2025.

3. M. Castro, B. Liskov, *Practical Byzantine Fault Tolerance*, OSDI, 1999.

4. A. Vaswani et al., *Attention Is All You Need*, NeurIPS, 2017.

5. BIP39 — Mnemonic code for generating deterministic keys.

InferenceChain is a research prototype. Not for production use.